

ASSESSMENT TOOL - 1

HACKER RANKER

NAME : PRIYADHARSHINI K

REG NO : 192421236

COURSE : CSA0927 – PROGRAMMING IN JAVA

HackerRank

Prepare > Java > Introduction Welcome to Java!

Exit Full Screen View

Problem

Submissions

Leaderboard

Welcome to the world of Java! In this challenge, we practice printing to stdout.

The code stubs in your editor declare a Solution class and a main method. Complete the main method by copying the two lines of code below and pasting them inside the body of your main method.

```
System.out.println("Hello, World.");
System.out.println("Hello, Java.");
```

Input Format

There is no input for this challenge.

Output Format

You must print two lines of output:

1. Print Hello, World. on the first line.
2. Print Hello, Java. on the second line.

Sample Output

```
Hello, World.
Hello, Java.
```

Change Theme Language Java 7

```
1 public class Solution {
2
3     public static void main(String[] args) {
4         System.out.println("Hello, World.");
5         System.out.println("Hello, Java.");
6     }
7 }
8
```

Line: 8 Col: 1

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Your Output (stdout)

```
1 Hello, World.
2 Hello, Java.
```

Expected Output

```
1 Hello, World.
2 Hello, Java.
```

HackerRank

Prepare > Java > Introduction > Java Stdin and Stdout I

Exit Full Screen View

Problem

Most HackerRank challenges require you to read input from `stdin` (standard input) and write output to `stdout` (standard output).

One popular way to read input from `stdin` is by using the `Scanner` class and specifying the Input Stream as `System.in`. For example:

```
Scanner scanner = new Scanner(System.in);
String myString = scanner.next();
int myInt = scanner.nextInt();
scanner.close();
```

The code above creates a `Scanner` object named `scanner` and uses it to read a `String` and an `int`. It then closes the `Scanner` object because there is no more input to read, and prints to `stdout` using `System.out.println(String)`. So, if our input is:

Hi 5

Our code will print:

myString is: Hi

Submissions

Leaderboard

Change Theme Language Java 7

```
1 import java.util.*;
2
3 public class Solution {
4
5     public static void main(String[] args) {
6         Scanner scan = new Scanner(System.in);
7         int a = scan.nextInt();
8         int b = scan.nextInt();
9         int c = scan.nextInt();
10
11         System.out.println(a);
12         System.out.println(b);
13         System.out.println(c);
14     }
15 }
16
```

Line: 16 Col: 1

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

```
1 42
2 100
3 125
```

[Download](#)

Your Output (stdout)

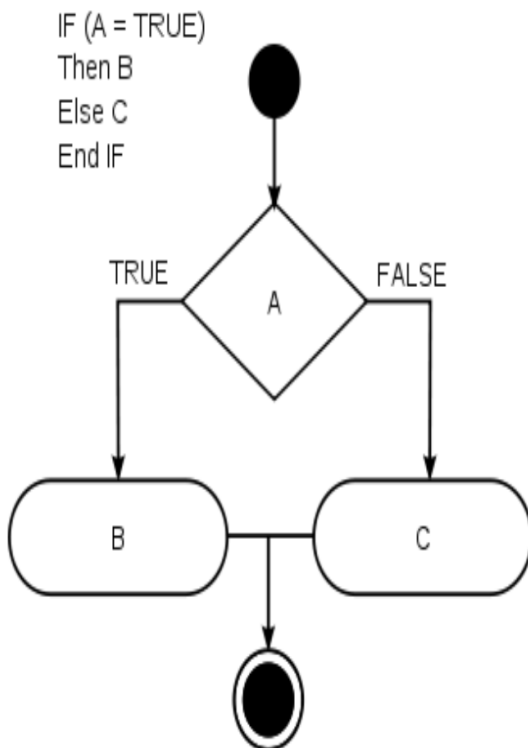
```
1 42
2 100
3 125
```

Expected Output

```
1 42
2 100
```

[Download](#)

In this challenge, we test your knowledge of using if-else conditional statements to automate decision-making processes. An if-else statement has the following logical flow:



[Change Theme](#) Language Java 7

```
1 import java.io.*;
2 import java.math.*;
3 import java.security.*;
4 import java.text.*;
5 import java.util.*;
6 import java.util.concurrent.*;
7 import java.util.regex.*;
8
9 public class Solution {
10
11     private static final Scanner scanner = new Scanner(System.in);
12
13     public static void main(String[] args) {
14         int N = scanner.nextInt();
15         scanner.skip("(\\r\\n|\\[\\n\\r\\u2028\\u2029\\u0085]?");
16
17         if (N % 2 != 0) {
18             // N is odd
19             System.out.println("Weird");
20         } else if (N >= 2 && N <= 5) {
21             // N is even and in range [2, 5]
22             System.out.println("Not Weird");
23         } else if (N >= 6 && N <= 20) {
```

Li

[Upload Code as File](#)

☐ Test against custom input

[Run Code](#)

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

[Download](#)

✓ Sample Test case 1

1 3

Your Output (stdout)

1 **Weird**

Expected Output

1 **Weird**

[Download](#)

In this challenge, you must read an integer, a double, and a String from stdin, then print the values according to the instructions in the Output Format section below. To make the problem a little easier, a portion of the code is provided for you in the editor.

Note: We recommend completing [Java Stdin and Stdout I](#) before attempting this challenge.

Input Format

There are three lines of input:

1. The first line contains an integer.
2. The second line contains a double.
3. The third line contains a String.

Output Format

There are three lines of output:

1. On the first line, print `String`: followed by the unaltered String read from stdin.
2. On the second line, print `Double`: followed by the unaltered double read from stdin.

Change Theme Language Java 7

```
1 import java.util.Scanner;
2
3 public class Solution {
4
5     public static void main(String[] args) {
6         Scanner scan = new Scanner(System.in);
7         int i = scan.nextInt();
8         double d = scan.nextDouble();
9         scan.nextLine(); // consume the leftover newline
10        String s = scan.nextLine();
11
12        System.out.println("String: " + s);
13        System.out.println("Double: " + d);
14        System.out.println("Int: " + i);
15    }
16 }
17
```

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Sample Test case 1

Input (stdin)

[Download](#)

```
1 42
2 3.1415
3 Welcome to HackerRank's Java tutorials!
```

Your Output (stdout)

```
1 String: Welcome to HackerRank's Java tutorials!
2 Double: 3.1415
3 Int: 42
```

Expected Output

[Download](#)

```
1 String: Welcome to HackerRank's Java tutorials!
2 Double: 3.1415
```

Java's `System.out.printf` function can be used to print formatted output. The purpose of this exercise is to test your understanding of formatting output using `printf`.

To get you started, a portion of the solution is provided for you in the editor; you must format and print the input to complete the solution.

Input Format

Every line of input will contain a String followed by an integer.

Each String will have a maximum of 10 alphabetic characters, and each integer will be in the inclusive range from 0 to 999.

Output Format

In each line of output there should be two columns:

The first column contains the String and is left justified using exactly 15 characters.

The second column contains the integer, expressed in exactly 3 digits; if the original input has less than three digits, you must pad your output's leading digits with zeroes.

Sample Input

```
java 100
cpp 65
python 50
```

[Change Theme](#) Language Java 7

```
1 import java.util.Scanner;
2
3 public class Solution {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         System.out.println("=====");
8         for (int i = 0; i < 3; i++) {
9             String s1 = sc.next();
10            int x = sc.nextInt();
11            System.out.printf("%-15s%03d\n", s1, x);
12        }
13        System.out.println("=====");
14    }
15 }
16
```

Line: 16 Col: 1

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✔ Sample Test case 0

✔ Sample Test case 1

Input (stdin)

```
1 java 100
2 cpp 65
3 python 50
```

Your Output (stdout)

```

1  =====
2  java          100
3  cpp           065
4  python        050
5  =====

```

HackerRank

PrepareJava > IntroductionJava Loops I

Exit Full Screen View

Problem

Objective

In this challenge, we're going to use loops to help us do some simple math.

Task

Given an integer, N , print its first 10 multiples. Each multiple $N \times i$ (where $1 \leq i \leq 10$) should be printed on a new line in the form: $N \times i = \text{result}$.

Input Format

A single integer, N .

Constraints

- $2 \leq N \leq 20$

Output Format

Print 10 lines of output; each line i (where $1 \leq i \leq 10$) contains the result of $N \times i$ in the form:
 $N \times i = \text{result}$.

Sample Input

2

Sample Output

Change ThemeLanguageJava 7

```
1 import java.io.*;  
2  
3 public class Solution {  
4     public static void main(String[] args) throws IOException {  
5         BufferedReader br = new BufferedReader(new InputStreamReader(System.in));  
6         int N = Integer.parseInt(br.readLine().trim());  
7  
8         for (int i = 1; i <= 10; i++) {  
9             System.out.println(N + " x " + i + " = " + (N * i));  
10        }  
11    }  
12 }  
13 
```

Line: 13 Col: 1

Upload Codes as File

Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

1 2

Your Output (stdout)

```
1 2 x 1 = 2
2 2 x 2 = 4
3 2 x 3 = 6
4 2 x 4 = 8
5 2 x 5 = 10
6 2 x 6 = 12
7 2 x 7 = 14
8 2 x 8 = 16
9 2 x 9 = 18
```

Problem

Java has 8 primitive data types; char, boolean, byte, short, int, long, float, and double. For this exercise, we'll work with the primitives used to hold integer values (byte, short, int, and long):

- A byte is an 8-bit signed integer.
- A short is a 16-bit signed integer.
- An int is a 32-bit signed integer.
- A long is a 64-bit signed integer.

Given an input integer, you must determine which primitive data types are capable of properly storing that input.

To get you started, a portion of the solution is provided for you in the editor.

Reference: <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html>

Input Format

The first line contains an integer, T , denoting the number of test cases.

Each test case, T , is comprised of a single line with an integer, n , which can be arbitrarily large or small.

Output Format

For each input variable n , and appropriate primitive *dataType*, you must determine if the given primitives are capable of storing it. If yes, then print:

Submissions

Leaderboard

Change Theme Language Java 7

```
1 import java.util.*;
2 import java.io.*;
3
4
5
6 class Solution{
7     public static void main(String []argh)
8     {
9
10
11
12         Scanner sc = new Scanner(System.in);
13         int t=sc.nextInt();
14
15         for(int i=0;i<t;i++)
16         {
17
18             try
19             {
20                 long x=sc.nextLong();
21                 System.out.println(x+" can be fitted in:");
22                 if(x>=-128 && x<=127)System.out.println("    byte");
23                 //Complete the code
```

Line: 40 Col: 1

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✔ Sample Test case 0

Your Output (stdout)

[illegible]

HackerRank

PrepareJavaIntroductionJava End-of-file

Exit Full Screen View

Problem

"In computing, End Of File (commonly abbreviated EOF) is a condition in a computer operating system where no more data can be read from a data source." — (Wikipedia: End-of-file)

The challenge here is to read n lines of input until you reach EOF, then number and print all n lines of content.

Hint: Java's Scanner.hasNext() method is helpful for this problem.

Input Format

Read some unknown n lines of input from stdin(System.in) until you reach EOF; each line of input contains a non-empty String.

Output Format

For each line, print the line number, followed by a single space, and then the line content received as input.

Sample Input

```
Hello world
I am a file
Read me until end-of-file.
```

Sample Output

Change ThemeLanguageJava 7

```
1 import java.io.*;
2 import java.util.*;
3 import java.text.*;
4 import java.math.*;
5 import java.util.regex.*;
6
7 public class Solution {
8
9     public static void main(String[] args) {
10         Scanner scanner = new Scanner(System.in);
11         int lineNumber = 1;
12         while (scanner.hasNextLine()) {
13             String line = scanner.nextLine();
14             System.out.println(lineNumber + " " + line);
15             lineNumber++;
16         }
17         scanner.close();
18     }
19 }
20
```

Line: 20 Col: 1

Upload Code as File

Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

```
1 Hello world
2 I am a file
3 Read me until end-of-file.
```

Your Output (stdout)

```
1 1 Hello world
2 2 I am a file
3 3 Read me until end-of-file.
```

Expected Output

```
1 1 Hello world
2 2 I am a file
```

HackerRank

PrepareJava > IntroductionJava Static Initializer Block

Exit Full Screen View

Problem

Submissions

Leaderboard

Static initialization blocks are executed when the class is loaded, and you can initialize static variables in those blocks.

It's time to test your knowledge of Static initialization blocks. You can read about it [here](#).

You are given a class `Solution` with a main method. Complete the given code so that it outputs the area of a parallelogram with breadth B and height H . You should read the variables from the standard input.

If $B \leq 0$ or $H \leq 0$, the output should be "java.lang.Exception: Breadth and height must be positive" without quotes.

Input Format

There are two lines of input. The first line contains B : the breadth of the parallelogram. The next line contains H : the height of the parallelogram.

Constraints

- $-100 \leq B \leq 100$
- $-100 \leq H \leq 100$

Output Format

If both values are greater than zero, then the main method must output the area of the parallelogram. Otherwise, print "java.lang.Exception: Breadth and height must be positive"

Change ThemeLanguageJava 7

1 > import java.io.*; ...

8 static int B, H;

9 static boolean flag = true;

10

11 static {

12 Scanner sc = new Scanner(System.in);

13 B = sc.nextInt();

14 H = sc.nextInt();

15 if (B <= 0 || H <= 0) {

16 flag = false;

17 System.out.println("java.lang.Exception: Breadth and height must be positive");

18 }

19 }

20

21 > public static void main(String[] args){ ...

Line: 8 Col: 1

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

✓ Sample Test case 1

Input (stdin)

1	1
2	3

Your Output (stdout)

1	3
---	---

Expected Output

1	3
---	---

HackerRank | Prepare Java > Introduction Java Int to String

Exit Full Screen View

Problem

You are given an integer n , you have to convert it into a string.

Please complete the partially completed code in the editor. If your code successfully converts n into a string s the code will print "Good job". Otherwise it will print "Wrong answer".

n can range between -100 to 100 inclusive.

Sample Input 0

100

Sample Output 0

Good job

Submissions

Leaderboard

Change Theme Language Java 7

```
4 import java.util.*;...
13
14 String s = String.valueOf(n);
15
16 if (n == Integer.parseInt(s)) {
17     System.out.println("Good job");
18 } else {
19     System.out.println("Wrong answer.");
20 }
21 catch (DoNotTerminate.ExitTrappedException e) {
22     System.out.println("Unsuccessful Termination!!");
23 }
24 }
25 }
26
27 //The following class will prevent you from terminating the code using exit(0)!
28 class DoNotTerminate {
29
30     public static class ExitTrappedException extends SecurityException {
31
32         private static final long serialVersionUID = 1;
33     }
34 }
```

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

1 100

Your Output (stdout)

1 Good job

Expected Output

1 Good job

The `Calendar` class is an abstract class that provides methods for converting between a specific instant in time and a set of calendar fields such as YEAR, MONTH, DAY_OF_MONTH, HOUR, and so on, and for manipulating the calendar fields, such as getting the date of the next week.

You are given a date. You just need to write the method, `getDay`, which returns the day on that date. To simplify your task, we have provided a portion of the code in the editor.

Example

`month = 8`

`day = 14`

`year = 2017`

The method should return `MONDAY` as the day on that date.

AUGUST 2017						
SUN	MON	TUE	WED	THU	FRI	SAT
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	31		

Change Theme Language Java 7

```
1 > import java.io.*; ...
8 class Result {
9
10     public static String findDay(int month, int day, int year) {
11         Calendar cal = Calendar.getInstance(TimeZone.getTimeZone("UTC"));
12         cal.set(year, month - 1, day); // month is 0-based in Calendar
13         int dow = cal.get(Calendar.DAY_OF_WEEK);
14
15         String[] names = {"SUNDAY", "MONDAY", "TUESDAY", "WEDNESDAY", "THURSDAY",
16                           "FRIDAY", "SATURDAY"};
17         return names[dow - 1];
18     }
19
20 > public class Solution { ...
```

Line: 8 Col: 1

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✔ Sample Test case 0

Input (stdin)

1	08 05 2015
---	------------

Your Output (stdout)

1 WEDNESDAY

Expected Output

1 WEDNESDAY

HackerRank

Prepare

Java

Introduction

Java Currency Formatter

Exit Full Screen View

Problem

Submissions

Leaderboard

Given a *double-precision* number, *payment*, denoting an amount of money, use the `NumberFormat` class' `getCurrencyInstance` method to convert *payment* into the US, Indian, Chinese, and French currency formats. Then print the formatted values as follows:

US: formattedPayment

India: formattedPayment

China: formattedPayment

France: formattedPayment

where *formattedPayment* is *payment* formatted according to the appropriate *Locale*'s currency.

Note: India does not have a built-in *Locale*, so you must *construct one* where the language is `en` (i.e., English).

Input Format

A single double-precision number denoting *payment*.

Constraints

- $0 \leq \text{payment} \leq 10^9$

Output Format

On the first line print US, then on the second line print formatted for US currency,

Change Theme

Language

Java 7

```

1 import java.util.*;
2 import java.text.*;
3
4 public class Solution {
5
6     public static void main(String[] args) {
7         Scanner scanner = new Scanner(System.in);
8         double payment = scanner.nextDouble();
9         scanner.close();
10
11         Locale usLocale = Locale.US;
12         Locale indiaLocale = new Locale("en", "IN");
13         Locale chinaLocale = Locale.CHINA;
14         Locale franceLocale = Locale.FRANCE;
15
16         NumberFormat usFormat = NumberFormat.getCurrencyInstance(usLocale);
17         NumberFormat indiaFormat = NumberFormat.getCurrencyInstance(indiaLocale);
18         NumberFormat chinaFormat = NumberFormat.getCurrencyInstance(chinaLocale);
19         NumberFormat franceFormat = NumberFormat.getCurrencyInstance(franceLocal
20
21         String us = usFormat.format(payment);
22         String india = indiaFormat.format(payment);
23         String china = chinaFormat.format(payment);

```

Line: 32 Col: 1

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

```
1 12324.134
```

Your Output (stdout)

```
1 US: $12,324.13
2 India: Rs.12,324.13
3 China: ¥12,324.13
4 France: 12 324,13 €
```

Expected Output

```
1 US: $12,324.13
2 India: Rs.12,324.13
3 China: ¥12,324.13
```

"A string is traditionally a sequence of characters, either as a literal constant or as some kind of variable." — Wikipedia: String (computer science)

This exercise is to test your understanding of Java Strings. A sample String declaration:

```
String myString = "Hello World!"
```

The elements of a String are called characters. The number of characters in a String is called the length, and it can be retrieved with the `String.length()` method.

Given two strings of lowercase English letters, *A* and *B*, perform the following operations:

1. Sum the lengths of *A* and *B*.
2. Determine if *A* is lexicographically larger than *B* (i.e.: does *B* come before *A* in the dictionary?).
3. Capitalize the first letter in *A* and *B* and print them on a single line, separated by a space.

Input Format

The first line contains a string *A*. The second line contains another string *B*. The strings are comprised of only lowercase English letters.

Change Theme Language Java 7

```
1 import java.io.*;
2 import java.util.*;
3
4 public class Solution {
5
6     public static void main(String[] args) {
7
8         Scanner sc=new Scanner(System.in);
9         String A=sc.next();
10        String B=sc.next();
11        /* Enter your code here. Print output to STDOUT. */
12        // 1) Sum of lengths
13        System.out.println(A.length() + B.length());
14
15        // 2) Lexicographical comparison
16        System.out.println(A.compareTo(B) > 0 ? "Yes" : "No");
17
18        // 3) Capitalize first letter of both strings and print
19        String cappedA = A.substring(0, 1).toUpperCase() + A.substring(1);
20        String cappedB = B.substring(0, 1).toUpperCase() + B.substring(1);
21
22        System.out.println(cappedA + " " + cappedB);
23    }
```

Line: 27 Col: 1

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

```
1 hello
2 java
```

Your Output (stdout)

```
1 9
2 No
3 Hello Java
```

Expected Output

```
1 9
2 No
3 Hello Java
```

Problem

We define the following terms:

- **Lexicographical Order**, also known as alphabetic or dictionary order, orders characters as follows:

$$A < B < \dots < Y < Z < a < b < \dots < y < z$$

For example, ball < cat, dog < dorm, Happy < happy, Zoo < ball.

- A **substring** of a string is a contiguous block of characters in the string. For example, the substrings of abc are a, b, c, ab, bc, and abc.

Given a string, *s*, and an integer, *k*, complete the function so that it finds the lexicographically smallest and largest substrings of length *k*.

Function Description

Complete the `getSmallestAndLargest` function in the editor below.

`getSmallestAndLargest` has the following parameters:

- string *s*: a string
- int *k*: the length of the substrings to find

Returns

- string: the string ' + "\n" + ' where and are the two substrings

Input Format

Submissions

Leaderboard

Change Theme

Language Java 7



```
1 > import java.util.Scanner;...
4     public static String getSmallestAndLargest(String s, int k) {
5         String smallest = "";
6         String largest = "";
7
8         for (int i = 0; i <= s.length() - k; i++) {
9             String sub = s.substring(i, i + k);
10
11             if (smallest.isEmpty() || sub.compareTo(smallest) < 0) {
12                 smallest = sub;
13             }
14             if (largest.isEmpty() || sub.compareTo(largest) > 0) {
15                 largest = sub;
16             }
17         }
18         return smallest + "\n" + largest;
19     }
20
21 > ...
```

Line: 7 Col: 9

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

```
1 welcometojava
2 3
```

Your Output (stdout)

```
1 ava
2 wel
```

Expected Output

```
1 ava
2 wel
```

A palindrome is a word, phrase, number, or other sequence of characters which reads the same backward or forward.

Given a string *A*, print Yes if it is a palindrome, print No otherwise.

Constraints

- A* will consist at most 50 lower case english letters.

Sample Input

madam

Sample Output

Yes

Change Theme Language Java 7

```
1 import java.util.Scanner;
2
3 public class Solution {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6         String s = sc.nextLine();
7
8         int i = 0, j = s.length() - 1;
9         boolean isPal = true;
10
11         while (i < j) {
12             if (s.charAt(i) != s.charAt(j)) {
13                 isPal = false;
14                 break;
15             }
16             i++;
17             j--;
18         }
19
20         System.out.println(isPal ? "Yes" : "No");
21         sc.close();
22     }
23 }
```

Line: 24 Col: :

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

1 **madam**

Your Output (stdout)

1 **Yes**

Expected Output

1 **Yes**

HackerRank

PrepareJava > StringsJava Anagrams

Exit Full Screen View

Problem

Submissions

Leaderboard

Two strings, a and b , are called anagrams if they contain all the same characters in the same frequencies. For this challenge, the test is not case-sensitive. For example, the anagrams of CAT are CAT, ACT, tac, TCA, aTC, and CtA.

Function Description

Complete the isAnagram function in the editor.

isAnagram has the following parameters:

- string a : the first string
- string b : the second string

Returns

- boolean: If a and b are case-insensitive anagrams, return true. Otherwise, return false.

Input Format

The first line contains a string a .

The second line contains a string b .

Constraints

- $1 \leq \text{length}(a), \text{length}(b) \leq 50$
- Strings a and b consist of English alphabetic characters.
- The comparison should NOT be case sensitive.

Change ThemeLanguageJava 7

```
1 > import java.util.Scanner; ...
4 static boolean isAnagram(String a, String b) {
5
6 if (a.length() != b.length()) return false;
7
8     a = a.toLowerCase();
9     b = b.toLowerCase();
10
11     int[] freq = new int[26]; // English alphabet
12
13     for (int i = 0; i < a.length(); i++) {
14         freq[a.charAt(i) - 'a']++;
15         freq[b.charAt(i) - 'a']--;
16     }
17
18     for (int count : freq) {
19         if (count != 0) return false;
20     }
21     return true;
22 }
23
24
25 > public static void main(String[] args) { ...
```

Line: 4 Col: 1

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

✓ Sample Test case 1

✓ Sample Test case 2

Input (stdin)

```
1 anagram
2 margana
```

Your Output (stdout)

```
1 Anagrams
```

Expected Output

```
1 Anagrams
```

HackerRank

PrepareJavaStringsJava String Tokens

Exit Full Screen View

Problem

Submissions

Leaderboard

Given a string, s , matching the regular expression $[A-Za-z !, ? , _ ' @]^+$, split the string into tokens. We define a token to be one or more consecutive English alphabetic letters. Then, print the number of tokens, followed by each token on a new line.

Note: You may find the `String.split` method helpful in completing this challenge.

Input Format

A single string, s .

Constraints

- $1 \leq \text{length of } s \leq 4 \cdot 10^5$
- s is composed of any of the following: English alphabetic letters, blank spaces, exclamation points (!), commas (,), question marks (?), periods (.), underscores (_), apostrophes ('), and at symbols (@).

Output Format

On the first line, print an integer, n , denoting the number of tokens in string s (they do not need to be unique). Next, print each of the n tokens on a new line in the same order as they appear in input string s .

Sample Input

```
anagram
margana
```

Change Theme

LanguageJava 7

⌵⌵⌵

```
1 import java.io.*;
2 import java.util.*;
3
4 public class Solution {
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7         String s = sc.nextLine();
8
9         String[] tokens = s.split("[A-Za-z]+");
10
11         int count = 0;
12         for (String t : tokens) {
13             if (!t.isEmpty()) count++;
14         }
15
16         System.out.println(count);
17         for (String t : tokens) {
18             if (!t.isEmpty()) System.out.println(t);
19         }
20     }
21 }
22
```

Line: 22 Col: 1

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

✓ Sample Test case 1

Input (stdin)

1 He is a very very good boy, isn't he?

Your Output (stdout)

1 10
2 He
3 is
4 a
5 very
6 very
7 good
8 boy
9 isn
10 t

HackerRank | Prepare | Java | Strings | Pattern Syntax Checker

Exit Full Screen View

Problem

Using **Regex**, we can easily match or search for patterns in a text. Before searching for a pattern, we have to specify one using some well-defined syntax.

In this problem, you are given a pattern. You have to check whether the syntax of the given pattern is valid.

Note: In this problem, a regex is only valid if you can compile it using the [Pattern.compile](#) method.

Input Format

The first line of input contains an integer N , denoting the number of test cases. The next N lines contain a string of any printable characters representing the pattern of a regex.

Output Format

For each test case, print `Valid` if the syntax of the given pattern is correct. Otherwise, print `Invalid`. Do not print the quotes.

Sample Input

```
3
([A-Z])(.+)
[AZ[a-z](a-z)
batcatpat(nat
```

Submissions

Leaderboard

Change Theme Language Java 7

```
1 import java.util.Scanner;
2 import java.util.regex.*;
3
4 public class Solution
5 {
6     public static void main(String[] args){
7         Scanner in = new Scanner(System.in);
8         int testCases = Integer.parseInt(in.nextLine());
9         while(testCases>0){
10             String pattern = in.nextLine();
11             //Write your code
12
13             try {
14                 Pattern.compile(pattern);
15                 System.out.println("Valid");
16             } catch (PatternSyntaxException e) {
17                 System.out.println("Invalid");
18             }
19             testCases--;
20         }
21     }
22 }
23
```

Line: 26 Col: 1

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

```
1 3
2 ([A-Z])(.+)
3 [AZ[a-z](a-z)
4 batcatpat(nat
```

Your Output (stdout)

```
1 Valid
2 Invalid
3 Invalid
```

HackerRank

Prepare Java > Data Structures Java Sort

Exit Full Screen View

Problem

You are given a list of student information: ID, FirstName, and CGPA. Your task is to rearrange them according to their CGPA in decreasing order. If two student have the same CGPA, then arrange them according to their first name in alphabetical order. If those two students also have the same first name, then order them according to their ID. No two students have the same ID.

Hint: You can use comparators to sort a list of objects. See the [oracle docs](#) to learn about comparators.

Input Format

The first line of input contains an integer N , representing the total number of students.

The next N lines contains a list of student information in the following structure:

ID Name CGPA

Constraints

$2 \leq N \leq 1000$

$0 \leq ID \leq 100000$

$5 \leq |Name| \leq 30$

$0 \leq CGPA \leq 4.00$

The name contains only lowercase English letters. The ID contains only integer numbers

Submissions

Leaderboard

Change Theme Language Java 7

```
1 import java.util.*;
2
3 class Student{
4     private int id;
5     private String fname;
6     private double cgpa;
7     public Student(int id, String fname, double cgpa) {
8         super();
9         this.id = id;
10        this.fname = fname;
11        this.cgpa = cgpa;
12    }
13    public int getId() {
14        return id;
15    }
16    public String getFname() {
17        return fname;
18    }
19    public double getGpa() {
20        return cgpa;
21    }
22 }
23
```

Line: 66 Col: 1

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✔ Sample Test case 0

11. [How to use a compass](#)

1	5
2	33 Rumpa 3.68
3	85 Ashis 3.85
4	56 Samiha 3.75
5	19 Samara 3.75
6	22 Fahim 3.76

Your Output (stdout)

1	Ashis
2	Fahim
3	Samara
4	Samiha
5	Rumpa

HackerRank |
 Prepare >
 Java >
 Advanced >
 Covariant Return Types
Exit Full Screen View

Java allows for [Covariant Return Types](#), which means you can vary your return type as long you are returning a subclass of your specified return type.

[Method Overriding](#) allows a subclass to override the behavior of an existing superclass method and specify a return type that is some subclass of the original return type. It is best practice to use the [@Override annotation](#) when overriding a superclass method.

Implement the classes and methods detailed in the diagram below:

```

classDiagram
    class Super {
        +Class Flower
        +Method String whatsYourName()
        +Return "I have many names and types."
    }
    class SubOne {
        +Class Jasmine
        +Method String whatsYourName()
        +Return "Jasmine"
    }
    class SubTwo {
        +Class Lotus
        +Method String whatsYourName()
        +Return "Lotus"
    }
    Super --|> SubOne
    Super --|> SubTwo

    class Super2 {
        +Class Region
        +Method Flower getNationalFlower()
        +Return An instance of Flower.
    }
    class SubThree {
        +Class WestBengal
        +Method Jasmine getNationalFlower()
        +Return An instance of Jasmine
    }
    class SubFour {
        +Class AndhraPradesh
        +Method Lily getNationalFlower()
        +Return An instance of Lily
    }
    Super2 --|> SubThree
    Super2 --|> SubFour
            
```

You will be given a partially completed code in the editor where the main method takes the name of a state (i.e., West Bengal or Andhra Pradesh) and prints the national

Change Theme
Language Java 7
+0

```

1 > import java.io.BufferedReader; ...
4 //Complete the classes
5
6 class Flower {
7     public String whatsYourName() {
8         return "";
9     }
10 }
11
12 class Jasmine extends Flower {
13     @Override
14     public String whatsYourName() {
15         return "Jasmine";
16     }
17 }
18
19 class Lily extends Flower {
20     @Override
21     public String whatsYourName() {
22         return "Lily";
23     }
24 }
25
            
```

Line: 4 Col: 1

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

```
1 AndhraPradesh
```

Your Output (stdout)

```
1 Lily
```

Expected Output

```
1 Lily
```

HackerRank

PrepareJavaData StructuresJava Subarray

Exit Full Screen View

Problem

Submissions

Leaderboard

We define the following:

- A subarray of an n -element array is an array composed from a contiguous block of the original array's elements. For example, if $array = [1, 2, 3]$, then the subarrays are $[1]$, $[2]$, $[3]$, $[1, 2]$, $[2, 3]$, and $[1, 2, 3]$. Something like $[1, 3]$ would not be a subarray as it's not a contiguous subsection of the original array.
- The sum of an array is the total sum of its elements.
 - An array's sum is negative if the total sum of its elements is negative.
 - An array's sum is positive if the total sum of its elements is positive.

Given an array of n integers, find and print its number of negative subarrays on a new line.

Input Format

The first line contains a single integer, n , denoting the length of array

$A = [a_0, a_1, \dots, a_{n-1}]$.

The second line contains n space-separated integers describing each respective element, a_i , in array A .

Constraints

- $1 \leq n \leq 100$
- $-10^4 \leq a_i \leq 10^4$

Change ThemeLanguageJava 7

```
7 public class Solution {
9     public static void main(String[] args) {
12         int n = scan.nextInt();
13         int[] a = new int[n];
14         for (int i = 0; i < n; i++) {
15             a[i] = scan.nextInt();
16         }
17
18         int count = 0;
19         for (int i = 0; i < n; i++) {
20             long sum = 0;
21             for (int j = i; j < n; j++) {
22                 sum += a[j];
23                 if (sum < 0) {
24                     count++;
25                 }
26             }
27         }
28
29         System.out.println(count);
30     }
31 }
32
```

Line: 32 Col: 1

Congratulations

You solved this challenge. Would you like to challenge your friends?

✓ Test case 0

✓ Test case 1 

✓ Test case 2 

✓ Test case 3 

✓ Test case 4 

✓ Test case 5 

Compiler Message

Success

Input (stdin)

```
1 5
2 1 -2 4 -5 1
```

Expected Output

```
1 9
```

HackerRank | Prepare Java Data Structures Java ArrayList

Exit Full Screen View

Problem

Sometimes it's better to use dynamic size arrays. Java's [ArrayList](#) can provide you this feature. Try to solve this problem using ArrayList.

You are given n lines. In each line there are zero or more integers. You need to answer a few queries where you need to tell the number located in y^{th} position of x^{th} line.

Take your input from `System.in`.

Input Format

The first line has an integer n . In each of the next n lines there will be an integer d denoting number of integers on that line and then there will be d space-separated integers. In the next line there will be an integer q denoting number of queries. Each query will consist of two integers x and y .

Constraints

- $1 \leq n \leq 20000$
- $0 \leq d \leq 50000$
- $1 \leq q \leq 1000$
- $1 \leq x \leq n$

Each number will fit in signed integer.

Total number of integers in n lines will not cross 10^5 .

Submissions

Leaderboard

Change Theme Language Java 7

```
7 public class Solution {
9     public static void main(String[] args) {
24     }
25
26     int q = Integer.parseInt(sc.nextLine().trim());
27
28     StringBuilder sb = new StringBuilder();
29     for (int i = 0; i < q; i++) {
30         StringTokenizer st = new StringTokenizer(sc.nextLine());
31         int x = Integer.parseInt(st.nextToken());
32         int y = Integer.parseInt(st.nextToken());
33
34         if (x >= 1 && x <= lines.size() && y >= 1 && y <= lines.get(x - 1).
35             ()) {
36             sb.append(lines.get(x - 1).get(y - 1)).append("\n");
37         } else {
38             sb.append("ERROR!").append("\n");
39         }
40     }
41     System.out.print(sb);
42 }
```

✓ Test case 0

✓ Test case 1

✓ Test case 2 

✓ Test case 3 

✓ Test case 4 

✓ Test case 5 

7	5
8	1 3
9	3 4
10	3 1
11	4 3
12	5 5

Expected Output

1	74
2	52
3	37
4	ERROR!
5	ERROR!

HackerRank | Prepare | Java | Data Structures | Java 2D Array

Exit Full Screen View

Problem

You are given a 6×6 2D array. An hourglass in an array is a portion shaped like this:

```
a b c
d
e f g
```

For example, if we create an hourglass using the number 1 within an array full of zeros, it may look like this:

```
1 1 1 0 0 0
0 1 0 0 0 0
1 1 1 0 0 0
0 0 0 0 0 0
0 0 0 0 0 0
0 0 0 0 0 0
```

Actually, there are many hourglasses in the array above. The three leftmost hourglasses are the following:

```
1 1 1 1 1 0 1 0 0
1 0 0
1 1 1 1 1 0 1 0 0
```

Submissions

Leaderboard

Change Theme Language Java 7

```
4 public class Solution {
5     public static void main(String[] args) throws IOException {
12         arr[i][j] = Integer.parseInt(st.nextToken());
13     }
14 }
15
16 int maxSum = Integer.MIN_VALUE;
17
18 for (int row = 0; row < 4; row++) {
19     for (int col = 0; col < 4; col++) {
20         int sum = arr[row][col] + arr[row][col+1] + arr[row][col+2]
21             + arr[row+1][col+1]
22             + int maxSum - Solution.main(String[]) arr[row+2][col+
23
24         if (sum > maxSum) {
25             maxSum = sum;
26         }
27     }
28 }
29
30 System.out.println(maxSum);
31 }
32
```

Line: 32 Col:

Test case 0

Test case 1

Test case 2

Test case 3

Test case 4

Test case 5

Test case 6

Compiler Message
Success

HackerRank

Prepare

Java

Data Structures

Java 1D Array

Exit Full Screen View

Submission

Leaderboard

Discussions

Editorial

You are not responsible for printing any output to stdout. Locked code in the editor loops through array *a* and prints each sequential element on a new line.

Sample Input

```
5
10
20
30
40
50
```

Sample Output

```
10
20
30
40
50
```

Explanation

When we save each integer to its corresponding index in *a*, we get *a* = [10, 20, 30, 40, 50]. The locked code prints each array element on a new line from

Change Theme

Language

Java 7

↺

⋮

```
1 import java.util.*;
2
3 public class Solution {
4
5     public static void main(String[] args) {
6
7         Scanner scan = new Scanner(System.in);
8         int n = scan.nextInt();
9
10        int[] a = new int[n];
11        for (int i = 0; i < n; i++) {
12            a[i] = scan.nextInt();
13        }
14        scan.close();
15
16        // Prints each sequential element in array a
17        for (int i = 0; i < a.length; i++) {
18            System.out.println(a[i]);
19        }
20    }
21 }
```

Line: 21 Col: :

✓ **Test case 0**

Success

✓ **Test case 1** 

Input (stdin)

1	5
2	10
3	20
4	30
5	40
6	50

Expected Output

1	10
2	20